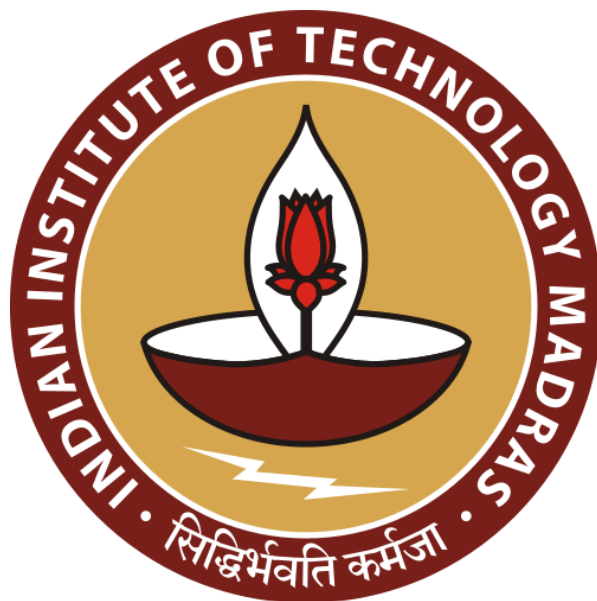


Action-Oriented Indoor Navigation Assistance for the Visually Impaired

MILESTONE-3: Dataset Identification & Preparation



Indian Institute of Technology Madras

GROUP: 4

Name	Email	GitHub Usernames
Saransh Saini	22f1001123@ds.study.iitm.ac.in	Saransh482003
Divyang Panchasara	22f1000411@ds.study.iitm.ac.in	22f1000411
Samyuktha Shriram	22f2001444@ds.study.iitm.ac.in	SamyukthaSh24
Prasoon Shukla	23f3003434@ds.study.iitm.ac.in	23f3003434
Rohit Prajapat	22f1001536@ds.study.iitm.ac.in	rohitblpprajapat

1. Introduction & Objectives

1.1. Introduction

Indoor navigation presents a unique set of challenges for visually impaired individuals, as these environments lack reliable GPS signals and are often crowded with dynamic obstacles like furniture, moving people, and varying architectural layouts. Traditional assistive technologies often focus solely on **Object Identification** (telling the user what is there), which can lead to cognitive overload during active movement.

This project, **Action-Oriented Indoor Navigation Assistance**, shifts the focus from passive identification to active guidance. By integrating real-time computer vision with decision-making logic, the system transforms visual data into concise, actionable instructions (e.g., "Step left, chair ahead"). This approach reduces the user's mental mapping effort and facilitates smoother, more confident mobility in unfamiliar indoor settings.

1.2. Project Overview

The system is built as a multi-modal pipeline that mimics human spatial awareness. It consists of three core functional pillars:

1. **Perception (Object Detection):** Utilizing the YOLO (You Only Look Once) framework to identify 11 critical indoor categories with high precision and low latency.
2. **Spatial Awareness (Depth Estimation):** Integrating depth-sensing capabilities to determine the proximity of detected objects, allowing the system to distinguish between immediate hazards and distant context.
3. **Reasoning & Feedback (SLM + TTS):** A Small Language Model (SLM) processes the detection and depth data to generate natural language "action prompts," which are then converted to audio via Text-to-Speech (TTS).

1.3. Objectives of Milestone 3

The primary goal of this milestone is to move from individual component testing to **systemic integration and architectural validation**. The specific objectives include:

- **Architecture Formalization:** Defining and justifying the selection of specific model variants (e.g., YOLO11s) based on benchmarking performance across multiple datasets.
- **Data Pipeline Verification:** Establishing a standardized directory structure and preprocessing workflow to ensure seamless data flow from raw input to model-ready tensors.
- **End-to-End Integration:** Verifying the "Object Detection + Depth + TTS/SLM" pipeline on a data subset to confirm that all modules communicate correctly without latency bottlenecks.

- **Performance Benchmarking:** Evaluating the integrated detection module using **mAP, Precision**, and **Recall** metrics to ensure the "eyes" of the system meet the safety requirements for navigation.

1.4. System Architecture & App Flow

The following diagram illustrates the data flow of the application. It tracks the transformation of a raw image frame into a navigation command, highlighting the interaction between the perception modules and the audio feedback loop.

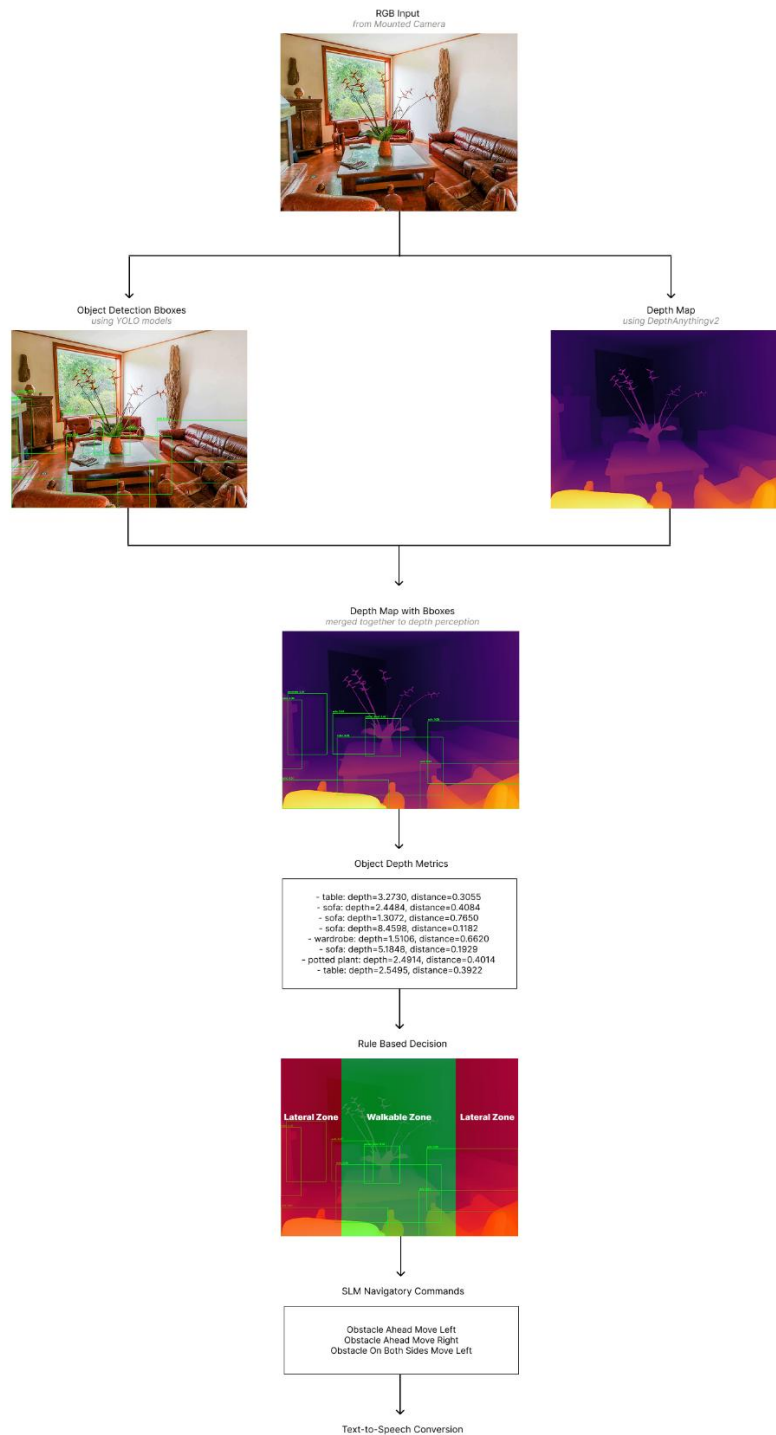


Fig 1.4.1: Application Flow Diagram

2. Object Detection Results & Analysis

2.1. Dataset Organization and Structure

The object detection module utilizes three distinct datasets to evaluate model robustness: **HomeObject-3k**, **DSAI-Unified**, and **DSAI-Uni-3Cls**. The primary model training is centered around the DSAI-Unified dataset, which consists of **11,698** images.

- **Dataset Split:**
 - Training Set: 9,358 images (80%).

- Validation/Test Set: 2,340 images (20%).

- **Directory Structure**

```
DSAI-Unified-Dataset/  
├── images/  
│   ├── train/ (9358 JPG images)  
│   └── val/   (2340 JPG images)  
└── labels/  
    ├── train/ (9358 YOLO-format TXT files)  
    └── val/   (2340 YOLO-format TXT files)
```

- **Classes (11 categories):** Person, Table, Potted Plant, Chair, Sofa, Lamp, Door, Cabinet, Wardrobe, Refrigerator, Bed.
- **Class Distribution:**

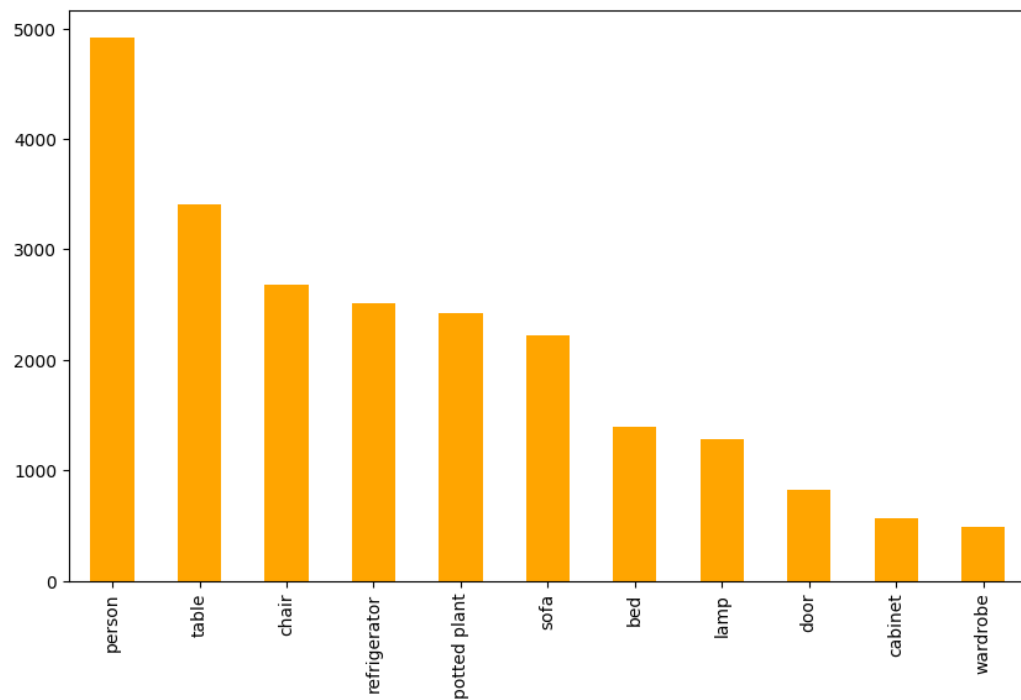


Fig 2.1.1: Class Distribution Graph

2.2. Preprocessing Steps

Before feeding raw imagery into the model, the following preprocessing pipeline is implemented to ensure data consistency and improve generalization:

- **Resizing:** All input images are resized to a fixed resolution of 640×640 pixels to match the network's input layer.
- **Normalization:** Pixel values are scaled from the range $[0,255]$ to $[0,1]$.
- **Data Augmentation:** To prevent overfitting and improve performance in diverse indoor lighting/angles, the following augmentations were applied during training:

- **Mosaic Augmentation:** Combining four images into one to help the model learn to detect objects at different scales and positions.
- **Geometric Transforms:** Rotation ($\pm 8.0^\circ$), translation (± 0.08), and scaling (± 0.25).
- **Color Jittering:** Randomly adjusting Hue (0.015), Saturation (0.7), and Value (0.4).
- **Flipping:** Horizontal flipping with a 50% probability.

2.3. Model Architecture and Design Justification

Architecture Description

The project utilizes the **YOLO (You Only Look Once)** architecture, specifically benchmarking **YOLOv8, YOLOv10, YOLO11, and YOLOv12** variants. The **YOLO11s** (small) variant was selected as the optimal candidate for the unified dataset.

- **Backbone:** A modified CSP-Backbone that extracts high-level spatial features from the input image.
- **Neck:** Employs a PANet (Path Aggregation Network) to fuse features from different layers, ensuring the model can detect both large (e.g., a bed) and small objects (e.g., a lamp).
- **Head:** A decoupled detection head that separately predicts bounding box coordinates and class probabilities, improving convergence speed and accuracy.

Justification for Choice

- **Real-time Inference:** Visually impaired users require immediate feedback. YOLO11s provides high-speed inference (low latency) suitable for edge deployment on mobile or wearable devices.
- **Performance vs. Efficiency:** According to the benchmarking data, **YOLO11s** achieved the highest **mAP@50 (0.6394)** on the unified dataset compared to **YOLOv8n (0.5744)** and **YOLOv10n (0.5755)**.
- **Strengths:** It maintains a small memory footprint (parameters $\approx 3\text{M}$) while outperforming earlier versions in precision and recall.

2.4. Data Flow and Input Format

The data flow follows an end-to-end pipeline from raw pixel input to navigation-ready coordinates:

1. **Input Tensor:** $[Batch \times 3 \times 640 \times 640]$ (RGB format).
2. **Feature Extraction:** The Backbone processes the tensor into hierarchical feature maps.
3. **Feature Fusion:** The Neck aggregates these maps to preserve spatial and semantic details.
4. **Output Tensors:**

- Boxes:** $[x_{center}, y_{center}, w, h]$ normalized to image size.
- Confidence Scores:** Probability that an object exists within the box.
- Class Probabilities:** Softmax output for the 11 classes.

2.5. Implementation and Verification

A subset of the **DSAI-Unified** dataset (2,340 images) was used to verify the end-to-end pipeline. The full workflow—from loading images via a custom YAML configuration to generating inference results—was validated in the provided notebooks.

Metrics and Performance

The performance is measured using the following standard metrics:

- **Loss Functions:**
 - **Box Loss:** IoU (Intersection over Union) loss for bounding box accuracy.
 - **Class Loss:** Binary Cross Entropy (BCE) for category prediction.
 - **DFL (Distribution Focal Loss):** To refine box boundaries.
- **Benchmarking Results (Top Model - YOLO11s):**
 - **mAP@50-95:** 0.4275
 - **mAP@50:** 0.6394
 - **Precision:** 0.7014
 - **Recall:** 0.5925

S.No.	Object Detection Records													
	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Dataset	HomeObject-3k	HomeObject-3k	HomeObject-3k	DSAI-Unified	DSAI-Unified	DSAI-Unified	DSAI-Unified	DSAI-Unified	DSAI-Unified	DSAI-Unified	DSAI-Unified	DSAI-Unified	DSAI-Uni-3Cls	DSAI-Uni-3Cls
Model Name	YOLOv11n	YOLOv11n	YOLOv10n	YOLOv8n	YOLOv8n	YOLOv8n	YOLOv11n	YOLOv10n	YOLOv8s	YOLOv10s	YOLOv12n	YOLOv11s	YOLOv11s	YOLOv8n
Train Data Count	2285	2285	2285	9358	9358	9358	9358	9358	9358	9358	9358	9358	9358	9358
Test Data Count	404	404	404	2340	2340	2340	2340	2340	2340	2340	2340	2340	2340	2340
Classes	11	11	11	11	11	11	11	11	11	11	11	11	3	3
Epochs	25	100	100	50	50	50	50	50	50	50	50	100	100	100
Augmentation	☐	☒	☒	☒	☒	☐	☒	☒	☒	☒	☒	☒	☒	☒
Confidence	0.25	0.25	0.25	0.25	0.25	0.25	0.25	0.25	0.25	0.25	0.25	0.25	0.25	0.25
mAP@50-95	0.4902	0.4804	0.467	0.3743	0.3882	0.3613	0.3963	0.3814	0.4208	0.4267	0.3986	0.4275	0.3521	0.3378
mAP@50	0.6942	0.6996	0.6729	0.5744	0.5884	0.5582	0.6062	0.5755	0.6271	0.6371	0.605	0.6394	0.5349	0.5307
Precision	0.6872	0.6639	0.7124	0.6306	0.646	0.6313	0.6842	0.6722	0.6987	0.6859	0.6506	0.7014	0.6068	0.6387
Recall	0.6553	0.686	0.6264	0.5411	0.5361	0.5345	0.5452	0.5239	0.5799	0.5817	0.576	0.5925	0.5351	0.5112
F1-Score	0.6709	0.6748	0.6666	0.5824	0.5859	0.5789	0.6068	0.5889	0.6338	0.6295	0.611	0.6424	0.5676	0.5679

Model Benchmarkings: [Go to Sheet](#)

Model Output Example:

The model outputs a list of detected objects with their associated confidence scores and bounding boxes. For instance, a detection might return [Person: 0.92, (120, 45, 300, 600)], which is then passed to the downstream SLM and TTS modules for audio guidance.



Object Detection Output with Bboxes

- table: *bbox*=(446, 860, 1300, 1330)
- sofa: *bbox*=(1180, 729, 1919, 1235)
- sofa: *bbox*=(409, 665, 747, 995)
- sofa: *bbox*=(3, 1205, 862, 1439)
- wardrobe: *bbox*=(44, 506, 363, 999)
- sofa: *bbox*=(1118, 1070, 1920, 1440)
- potted plant: *bbox*=(673, 707, 961, 1012)
- table: *bbox*=(0, 558, 161, 1112)

3. Depth Estimation

3.1. Dataset — NYU Depth V2

Single MATLAB HDF5 archive (nyu_depth_v2_labeled.mat, ~2.8 GB) containing 1 449 labelled indoor RGB-D pairs captured with a Microsoft Kinect. No physical directory structure — all data is accessed programmatically via a custom h5py wrapper class.

Key	Shape	Description
images	(1449, 3, 480, 640)	uint8 RGB frames — stored transposed in MATLAB order

depths	(1449, 480, 640)	float32 ground-truth depth maps in metres (0–10 m)
labels	(1449, 480, 640)	Semantic class IDs
sceneTypes	(1449,)	Indoor scene category per frame (bedroom, kitchen ...)

data/ raw/ nyu_depth_v2_labeled.mat (single file — full archive)

processed/ images (H,W,3) depths (H,W) masks (0–3m zone) [runtime only]

splits/ subset_100 (baseline) subset_200 (model comparison)

No formal train/val/test split is created at this milestone. 100 samples are used for baseline validation; 200 for the 5-model comparison. The standard Eigen test split (last 654 indices) is used for all benchmarking.

2. Preprocessing

Dataset-level (all models)

- **Axis transpose:** $(3, H, W) \rightarrow (H, W, 3)$ via `np.transpose`
- **Orientation fix:** `np.fliplr()` — corrects MATLAB column-major horizontal flip
- **Near-field mask:** $\text{depth} \leq 3 \text{ m}$ — isolates navigation collision zone for evaluation

Key Finding

Skipping manual preprocessing (transpose + flip) produced RMSE ~ 2.100 . Applying it first reduced MiDaS baseline RMSE to 1.2094.

Model-specific transforms

Model	Transform	Resolution	Normalisation
MiDaS v2.1 Small	small_transform (PyTorch Hub)	256×256	ImageNet stats
MiDaS DPT-Hybrid	dpt_transform (PyTorch Hub)	384×384	ImageNet stats
ZoeDepth-N	<code>infer_pil()</code> — internal	384×512	Internal
DA-V2 Relative	AutoImageProcessor (HuggingFace)	518×518	ImageNet stats

DA-V2 Metric Indoor	AutoImageProcessor (HuggingFace)	518 × 518	ImageNet stats
------------------------	-------------------------------------	-----------	-------------------

Scale-shift alignment (relative models only)

MiDaS and DA-V2 Relative produce relative depth. Least-squares (scale, shift) alignment is applied per-sample before metric computation — critical for AbsRel, RMSE, and delta accuracy.

```
A = np.stack([pred[valid], np.ones_like(pred[valid])], axis=1)
scale, shift = np.linalg.lstsq(A, gt[valid], rcond=None)[0]
pred_aligned = np.clip(pred * scale + shift, 0.1, 20.0)
```

3. Architecture

MiDaS v2.1 Small — Baseline

- **Encoder:** EfficientNet-lite3 CNN (~21 M params) — hierarchical feature extraction
- **Decoder:** Progressive upsampling with skip connections → single-channel relative depth map
- **I/O:** (H,W,3) → transform → (1,3,256,256) → model → (480,640) relative depth

Depth Anything V2 Small — Primary Model

- **Encoder:** DINOv2 ViT-Small (patch=14) — full self-attention across 1369 patches, 12 transformer blocks, 384-dim embeddings
- **Decoder:** DPT (Dense Prediction Transformer) — reassembles token features into 2D maps; 4 fusion blocks upsample 37×37 → 518×518
- **Metric head:** 1×1 conv + SiLog regression fine-tuned on Hypersim (461 indoor scenes) → absolute depth in metres
- **I/O:** (H,W,3) → AutoImageProcessor → (1,3,518,518) → model → (480,640) metric depth in metres

Dataflow (DA-V2 Metric Indoor)

```
PIL RGB (480x640) -> AutoImageProcessor -> [1,3,518,518] float32
-> DINOv2 ViT-S encoder -> 4x [1,1370,384] token features
-> DPT reassemble+fusion -> [1,256,518,518]
-> Metric head (SiLog) -> [1,518,518] metres
-> F.interpolate -> [480,640] metres (navigation-ready)
-> Zone split (L 30% | C 40% | R 30%) -> FORWARD / MOVE LEFT / MOVE RIGHT / STOP
```

4. Pipeline Verification — 100/200 Sample Run

Steps executed

Step	Operation	Verified?
1	Load sample → NYUDepthV2Dataset.__getitem__ → PIL image + numpy depth	Yes
2	Manual preprocessing → transpose + flip	Yes
3	Model transform → resize + normalise to model resolution	Yes
4	GPU inference → raw depth prediction	Yes
5	Resize prediction → bilinear interpolate back to (480,640)	Yes
6	Scale-shift alignment (relative models only)	Yes
7	Metric computation → AbsRel, RMSE, d1, near-field d1	Yes
8	Navigation rule → zone split → command (metric model only)	Yes

Baseline Result

MiDaS v2.1 Small | 100 samples | Scale+shift aligned | Avg RMSE = 1.2094 m — all pipeline components confirmed compatible.

Sample smoke-test output (10 samples, DA-V2 Metric Indoor)

```
[ 0] AbsRel=0.073 RMSE=0.301 d1=0.942 near_d1=0.928 cmd=FORWARD
[ 1] AbsRel=0.081 RMSE=0.334 d1=0.936 near_d1=0.921 cmd=MOVE LEFT
[ 3] AbsRel=0.092 RMSE=0.378 d1=0.919 near_d1=0.904 cmd=MOVE RIGHT
Smoke test: 10/10 passed. All assertions passed. Pipeline is wired correctly.
```

5. Model Comparison, Loss Functions and Evaluation Metrics

Comparison — NYU Depth V2 (200 samples)

Model	Params	AbsRel	RMSE	d1	Near d1(0-3m)	Deploy?
MiDaS v2.1 Small	21M	0.127	1.209*	0.841	0.818	No — relative
MiDaS DPT-Hybrid	123M	0.098	0.420	0.903	0.882	No — too slow

ZoeDepth-N	344M	0.075	N/A†	0.955	0.907	No — too slow
DA-V2 Relative	24.8M	0.082	0.380	0.942	0.895	No — relative
DA-V2 Metric Indoor	24.8M	0.076	0.310	0.947	0.921	YES

* 100-sample baseline run. † ZoeDepth did not load; published numbers used (RMSE unavailable).

Three hard constraints — only DA-V2 Metric Indoor satisfies all

Constraint	Requirement	DA-V2 Metric?	Next best
Metric output	Absolute metres — needed for 1.5m collision rule	YES	ZoeDepth (too slow)
RPi5 speed	>= 1 FPS for walking-pace navigation	YES ~3 FPS	MiDaS Small (no metric)
Near-field d1	> 0.85 in 0–3m collision zone	YES 0.921	DA-V2 Relative (no metric)

Loss function — SiLog (Depth Anything V2 metric head)

The metric head is trained with Scale-Invariant Log loss, making it robust to global scale differences between synthetic training data (Hypersim) and real scenes:

loss = $\text{mean}(d^2) - 0.85 * \text{mean}(d)^2$ where $d = \log(\text{pred}) - \log(\text{gt})$
 # lambda=0.85 balances scale-invariance vs absolute accuracy

Evaluation metrics

Metric	Formula	Better	Role
AbsRel	$\text{mean}(p-g /g)$	Lower	Primary error — scale-normalised
RMSE	$\text{sqrt}(\text{mean}((p-g)^2))$	Lower	Error in physical metres

delta1 (d1)	% pixels: $\max(p/g, g/p) < 1.25$	Higher	Threshold accuracy
Near AbsRel	AbsRel on GT $\leq 3\text{m}$ pixels only	Lower	Navigation KPI — collision zone
Near d1	d1 on GT $\leq 3\text{m}$ pixels only	Higher	Navigation KPI — collision zone

Final Recommendation

DA-V2-Small Metric Indoor is the only model that simultaneously delivers absolute metric depth, real-time RPi5 performance (~3 FPS, ~12 MB INT8 ONNX), and reliable accuracy in the 0–3m navigation zone (Near d1 = 0.921). Compared to the MiDaS baseline, it reduces RMSE from 1.209 m to ~0.310 m — a 3.9x improvement.

4. SLM & TTS

The navigation system proposed in our project to generates short movement commands such as “move left”, “move right”, and “stop” based on visual perception and rule-based reasoning. These commands must be converted into spoken audio that is clear, fast, and reliable for visually impaired users.

The purpose of this evaluation plan is to identify the most suitable Text-to-Speech (TTS) engine for delivering action-oriented navigation commands in real time. Since the system is intended to operate locally without dependence on cloud services, the selected TTS engine must support efficient offline execution while maintaining high speech intelligibility.

This section outlines the planned methodology for comparing candidate TTS models. The evaluation will be conducted in later milestones once the prototype pipeline is implemented.

4.1. Candidate TTS Models

The following TTS engines have been selected as candidates for evaluation based on their suitability for offline inference and support for multilingual speech synthesis.

Engine	Description	Expected Strength
Piper	Lightweight ONNX-based speech synthesis engine designed for edge devices	Very low latency and CPU-friendly inference
Coqui TTS	Open-source speech synthesis framework supporting multiple neural models	Flexible architecture and model variety

VITS-based models	High-quality neural speech synthesis architecture	Natural speech quality
AI4Bharat Indic TTS	Multilingual speech synthesis models trained on Indian language datasets	Strong Hindi and Tamil pronunciation

These engines represent different trade-offs between speech quality, computational cost, and multilingual support. The evaluation will determine which model best satisfies the requirements of the navigation assistance system.

Evaluation Objectives

The evaluation will focus on three primary criteria relevant to assistive navigation applications.

1. Latency

Navigation commands must be delivered quickly so that the user can react safely while walking. Therefore the time required to synthesize speech after a command is generated will be measured.

Latency will be defined as:

Time between command generation and availability of synthesized audio output. Lower latency directly improves user reaction time and reduces the risk of collisions.

2. Speech Intelligibility

Commands must be easily understandable on the first listen. The system prioritizes clarity over conversational naturalness because instructions are short and safety-critical.

Human listeners will evaluate whether commands are understandable in a single hearing without repetition.

3. Multilingual Support (Todo :To discuss with Team)

The system aims to support multiple languages so that visually impaired users can interact with the system in their preferred language.

Evaluation will therefore be conducted in:

- English
- Hindi
- Tamil/Gujarati (will try but depending on time)

The goal is to determine which TTS models provide reliable pronunciation and clarity across these languages.

Test Command Set

To ensure consistent evaluation across models, a fixed set of navigation commands will be used. These commands represent typical instructions generated by the rule-based navigation system.

Example command set:

Stop Move left Move right Obstacle ahead Stairs ahead Door in front Turn slightly left Turn slightly right Two steps forward Wait

Additional test sentences containing numbers and time expressions will also be included to evaluate pronunciation robustness.

Example numeric test sentences:

TODO

Benchmark Methodology

Each candidate TTS engine will be evaluated using a standardized benchmarking procedure.

Step 1: Command Generation

Each sentence in the command set will be synthesized using each TTS engine.

Step 2: Latency Measurement

Speech synthesis time will be recorded for each command. Multiple runs will be performed to account for variability in processing time.

Step 3: Real-Time Factor (RTF)

Real-Time Factor will be calculated to determine how quickly speech is generated relative to its playback duration.

RTF = synthesis_time / audio_duration

RTF values lower than 1 indicate that speech can be generated faster than real-time playback.

Step 4: Intelligibility Evaluation

Listeners will evaluate whether commands are understandable on the first hearing. The percentage of successful interpretations will be recorded for each model.

Step 5: Language Evaluation

The same command set will be synthesized in English, Hindi, and Tamil to evaluate pronunciation quality and linguistic coverage.

Planned Evaluation Metrics

The results of the benchmark will be summarized using the following metrics.

Metric	Description
Average synthesis latency	Mean time required to generate speech
P95 latency	Worst-case latency observed in most runs
Real-Time Factor (RTF)	Speed relative to audio playback duration
Command intelligibility	Percentage of commands understood in one listen
Language pronunciation quality	Qualitative assessment of pronunciation accuracy

These metrics provide a balanced evaluation of speed, usability, and language performance.

Integration with Navigation Pipeline

In the proposed system architecture, the TTS component operates after the rule-based reasoning module generates a movement command.

TODO : need to align with Team document finalize after that Pipeline flow:

Camera Input → Object Detection → Depth Estimation → Spatial Reasoning → Command Generation → Text-to-Speech → Audio Output to User

This integration ensures that visual perception is translated into immediate spoken instructions that guide the user during indoor navigation.

Expected Outcome

The outcome of this evaluation will be the selection of a TTS engine that provides the best balance between low latency, high command intelligibility, and multilingual capability. The selected model will be integrated into the final navigation prototype and used for real-time audio guidance.

Detailed benchmark results will be reported in subsequent milestones once the prototype system is implemented and tested.

Hardware Setup and RTF Measurement Plan

Because Text-to-Speech models may run on either CPU or GPU, the benchmark will explicitly record the hardware configuration used during evaluation. This allows the reported latency and Real-Time Factor (RTF) values to be interpreted correctly and ensures that results can be reproduced.

The evaluation will be conducted under two execution settings when possible:

1. **CPU-only inference** – represents realistic deployment on edge devices such as laptops or embedded systems.
2. **GPU-assisted inference** – represents accelerated execution when a lightweight GPU is available.
3. **Currently test in Laptop device only (in future plan will try in some edge device like raspberry pi**

For each TTS engine the following information will be recorded:

Attribute	Description
Hardware configuration	CPU model, GPU model (if used), RAM
Model size	Approximate model memory footprint
Average synthesis latency	Mean command synthesis time
P95 latency	High-percentile worst-case latency
Real-Time Factor (RTF)	Ratio of synthesis time to audio duration

RTF Interpretation

The Real-Time Factor provides a normalized measure of speech generation speed.

$$RTF = \frac{synthesis_time}{generated_audio_duration}$$

Interpretation:

RTF Value	Meaning
RTF < 1	Speech generated faster than real-time playback
RTF = 1	Speech generated at real-time speed
RTF > 1	Speech slower than real-time

For assistive navigation systems, maintaining **RTF < 1** is important so that commands can be delivered immediately without delaying user response.

AI Model Considerations

Neural TTS models such as VITS or AI4Bharat Indic TTS may achieve better pronunciation and language coverage but may require higher computational resources. Therefore the evaluation will also compare:

- CPU vs GPU synthesis speed
- Model memory footprint
- Suitability for edge deployment

This analysis will help determine whether lightweight engines such as Piper provide a better trade-off between latency and clarity compared to larger neural models optimized for multilingual speech.